

GESTIONE GITE SCOLASTICHE

Manuale d'Uso — Versione 1.0

GRUPPO 4:

Sbragi Niccolò · Barbagli Marco · Bucsai Alex · Bruschi Lorenzo

MySoft snc © 2026

1. Introduzione

Gestione Gite Scolastiche è un'applicazione desktop sviluppata in Java con interfaccia grafica Swing. Permette di gestire in modo integrato alunni, itinerari e prenotazioni per le uscite didattiche di una scuola.

Le funzionalità principali sono:

- Anagrafica alunni con gestione automatica del flag minorenne e autorizzazioni parentali
- Catalogo itinerari con destinazione (nome, regione, nazione), durata, tipo, costo e vincoli partecipanti
- Prenotazioni di gite scolastiche con validazione completa e calcolo acconto automatico (10%)
- Salvataggio automatico su file binari .dat alla chiusura tramite shutdown hook
- Caricamento automatico dei dati salvati all'avvio dell'applicazione

2. Requisiti di sistema

Campo	Descrizione
Sistema operativo	Windows 10/11, macOS 12+, Linux (qualsiasi distribuzione recente)
Java Runtime	JDK / JRE versione 8 o superiore
RAM	Minimo 256 MB disponibili
Disco	Almeno 10 MB liberi per i file dati (.dat)
Display	Risoluzione minima 1024 × 768 pixel

3. Avvio dell'applicazione

Per avviare il programma compilare ed eseguire la classe Main dal terminale:

```
javac *.java && java Main
```

All'avvio il sistema esegue automaticamente le seguenti operazioni:

1. Istanza i tre servizi: AlunnoService, ItinerarioService, PrenotazioneService
2. Carica i dati salvati dai file alunni.dat, itinerari.dat, prenotazioni.dat
3. Apre la finestra principale (MainFrame) con il pannello Gestione Alunni visibile di default
4. Registra un shutdown hook: alla chiusura della JVM i dati vengono salvati automaticamente

Nota: se i file .dat non esistono ancora (primo avvio), FileManager restituisce liste vuote senza mostrare errori; il programma parte correttamente con dati vuoti.

4. Interfaccia principale

La finestra principale (MainFrame, dimensione 900×600 px) è costruita con BorderLayout ed è divisa in due aree principali:

Area	Descrizione
Menu laterale (WEST)	Quattro pulsanti di navigazione: Gestione Alunni, Gestione Itinerari, Gestione Prenotazioni, Esci
Area centrale (CENTER)	Pannello a schede (CardLayout) che cambia contenuto in base al pulsante premuto; al lancio mostra la scheda "Gestione Alunni"

Il pulsante **Esci** mostra una finestra di conferma (JOptionPane YES/NO). Scegliendo 'Sì' viene chiamato `System.exit(0)`, che attiva lo shutdown hook e salva tutti i dati.

5. Gestione Alunni

Il pannello `PanelAlunni` permette di inserire nuovi alunni e visualizzare l'elenco completo. Utilizza `AlunnoService` per tutte le operazioni.

5.1 Campi del modulo

Campo	Tipo	Obbligatorio	Note
ID	Intero	Sì	Deve essere univoco; <code>AlunnoService.cercaPerId()</code> controlla i duplicati
Nome	Testo	Sì	Non può essere vuoto
Cognome	Testo	Sì	Non può essere vuoto
Età	Intero	Sì	Determina automaticamente il flag minorenni (< 18)
Classe	Testo	Sì	Es. '3A', '5B' — non può essere vuoto
Anno corso	Intero	Sì	Anno scolastico di riferimento
Minorenne	Checkbox	—	Calcolato automaticamente dall'età; non modificabile dall'utente

5.2 Come aggiungere un alunno

1. Inserire un ID numerico intero nel campo ID
2. Digitare nome, cognome, età, classe e anno corso
3. Premere il pulsante **Aggiungi**
4. Un messaggio JOptionPane confermerà l'inserimento; l'alunno appare nell'area di testo in basso
5. Usare **Pulisci** per svuotare il modulo senza modificare l'elenco

5.3 Logica del flag minorenni

Il flag *minorenne* viene calcolato nel costruttore di `Alunno` con l'espressione `eta < 18` e aggiornato automaticamente da `setEta()`. La checkbox nel pannello è disabilitata: l'utente non può modificarla direttamente.

5.4 Autorizzazione e rinuncia medica

I flag *autorizzazione* e *rinunciaMedica* sono gestiti tramite `AlunnoService.autorizzaAlunno(id)` e `AlunnoService.rinunciaMedica(id)`. Nella GUI attuale questi flag non sono modificabili direttamente dall'interfaccia grafica; la loro impostazione è disponibile via codice. **Attenzione:** un alunno minorenne senza autorizzazione non può essere inserito in una prenotazione (`PrenotazioneService` blocca la creazione).

5.5 Elenco alunni

All'avvio il pannello carica automaticamente gli alunni presenti in AlunnoService (già popolato da Main tramite FileManager) e li mostra nell'area di testo. Ogni alunno è visualizzato con: ID, Nome, Cognome, Età, Classe, Minorenne (SI/NO).

6. Gestione Itinerari

Il pannello PanellItinerario permette di creare nuovi itinerari e visualizzare quelli già inseriti. Ogni itinerario comprende una Destinazione (oggetto separato con nome, regione, nazione) e tutti i parametri del viaggio.

6.1 Campi del modulo

Campo	Tipo	Obbligatorio	Note
ID	Intero	Sì	Identificativo univoco dell'itinerario
Nome destinazione	Testo	Sì	Es. 'Roma', 'Firenze'
Regione	Testo	Sì	Regione della destinazione
Nazione	Testo	Sì	Nazione della destinazione
Giorni	Intero	Sì	Durata del viaggio in giorni
Tipo	Testo	Sì	Es. 'culturale', 'naturalistico', 'sportivo'
Costo (€)	Decimale	Sì	Costo per persona; separatore decimale: punto (es. 150.50)
Min partecipanti	Intero	Sì	Numero minimo; deve essere $\leq$ Max
Max partecipanti	Intero	Sì	Numero massimo partecipanti
Anno corso	Intero	Sì	Anno scolastico di riferimento
Descrizione	Testo	No	Testo descrittivo dell'itinerario
Optional	Testo	No	Servizi aggiuntivi inclusi

6.2 Come aggiungere un itinerario

1. Compilare tutti i campi obbligatori (Nome destinazione, Regione, Nazione, Tipo sono testo; gli altri numerici)
2. Verificare che Min partecipanti non superi Max partecipanti
3. Premere **Aggiungi**
4. ItinerarioService verifica la validità; se corretta, l'itinerario compare nell'area inferiore
5. Usare **Pulisci** per svuotare il modulo

6.3 Protezione degli itinerari prenotati

Una volta che un itinerario ha almeno una prenotazione attiva, il flag *prenotato* viene impostato a true da PrenotazioneService. Da quel momento ItinerarioService blocca qualsiasi tentativo di modifica della descrizione o di eliminazione, per garantire la coerenza con le prenotazioni esistenti.

6.4 Calcolo del costo totale itinerario

Il metodo **calcolaCostoTotaleItinerario()** di `Itinerario` restituisce $\text{costo} \times \text{numeroPartecipanti}$. Questo valore viene usato come riferimento da `PrenotazioneService` per il calcolo dell'acconto.

7. Gestione Prenotazioni

Il pannello `PanelPrenotazione` gestisce la creazione e l'annullamento delle prenotazioni. Collega alunni e itinerari già presenti nei rispettivi service.

7.1 Campi del modulo

Campo	Tipo	Obbligatorio	Note
ID Prenotazione	Intero	Sì	Deve essere univoco; verificato tramite <code>cercaPerId()</code>
ID Itinerario	Intero	Sì	L'itinerario deve esistere in <code>ItinerarioService</code>
ID Alunni	Testo	Sì	Lista di ID numerici separati da virgola (es. '1,2,3')

7.2 Come effettuare una prenotazione

1. Inserire un ID Prenotazione univoco
2. Inserire l'ID dell'itinerario desiderato (deve esistere nel pannello `Gestione Itinerari`)
3. Inserire gli ID degli alunni partecipanti separati da virgola
4. Premere **Prenota**
5. `PrenotazioneService` esegue le validazioni (vedi sezione 9); se superano, la prenotazione viene salvata
6. Un messaggio mostra l'acconto calcolato (10% del costo totale); il riepilogo appare nell'area di testo

Nota: il costo per persona viene letto direttamente dall'oggetto `Itinerario`; non è un valore fisso.

7.3 Calcolo automatico del costo

Voce	Formula
Costo totale prenotazione	<code>itinerario.getCosto() × numero partecipanti</code>
Acconto (10%)	$\text{Costo totale} \times 0.10$

7.4 Annullare una prenotazione

1. Premere il pulsante **Annulla prenotazione**
2. Inserire l'ID della prenotazione da annullare nella finestra di dialogo
3. Inserire il motivo dell'annullamento (campo obbligatorio)
4. Confermare: la prenotazione viene marcata come annullata tramite `Prenotazione annullaGita(motivo)`; nell'area di testo compare ***** PRENOTAZIONE [ID] ANNULLATA ***** con il motivo

Il motivo è obbligatorio per tracciare la causa. Se il campo è vuoto o l'ID non viene trovato, viene mostrato un messaggio di errore e l'operazione non avviene.

7.5 Elenco prenotazioni

All'apertura del pannello vengono caricate le prenotazioni salvate in sessioni precedenti. Per ciascuna vengono mostrati: ID Prenotazione, Destinazione, numero Partecipanti, Costo totale, Acconto 10%, Stato (ATTIVA / ANNULLATA).

8. Salvataggio e caricamento dati

L'applicazione gestisce la persistenza dei dati tramite serializzazione Java binaria (FileManager.java). Le classi Alunno, Itinerario, Prenotazione e Destinazione implementano Serializable.

File	Contenuto	Classe gestita
alunni.dat	Lista di tutti gli alunni registrati	FileManager.salvaAlunni / caricaAlunni
itinerari.dat	Lista di tutti gli itinerari inseriti	FileManager.salvaItinerari / caricaItinerari
prenotazioni.dat	Lista di tutte le prenotazioni	FileManager.salvaPrenotazioni / caricaPrenotazioni

- I file vengono creati nella stessa cartella da cui si avvia il programma
- Il salvataggio avviene automaticamente allo shutdown della JVM tramite `Runtime.getRuntime().addShutdownHook()`
- Il caricamento avviene automaticamente in `Main.main()` prima dell'avvio della GUI
- Se un file non esiste o è corrotto, FileManager restituisce una lista vuota; l'applicazione non si blocca
- **Non modificare manualmente i file .dat:** sono in formato binario e potrebbero corrompersi

9. Regole di validazione

`PrenotazioneService.creaPrenotazione()` applica i seguenti controlli nell'ordine indicato:

Controllo	Condizione	Comportamento se fallisce
Prenotazione non nulla	<code>p != null</code>	Stampa messaggio, esce senza salvare
Itinerario presente	<code>p.getItinerario() != null</code>	Stampa messaggio, esce
Minimo partecipanti	<code>partecipanti >= itinerario.getMinPartecipanti()</code>	Stampa messaggio, esce
Massimo partecipanti	<code>partecipanti <= itinerario.getMaxPartecipanti()</code>	Stampa messaggio, esce
Autorizzazione minorenni	Per ogni alunno minorenne: <code>isAutorizzatoMinorenne()</code>	Stampa messaggio, esce

Solo dopo aver superato tutti i controlli viene impostato `itinerario.setPrenotato(true)` e la prenotazione aggiunta alla lista.

Validazioni in AlunnoService

Operazione	Controllo
<code>aggiungiAlunno()</code>	L'oggetto Alunno non deve essere null

Operazione	Controllo
rimuoviAlunno()	L'ID deve corrispondere a un alunno esistente
autorizzaAlunno()	L'alunno deve esistere ed essere minorenne
rinunciaMedica()	L'alunno deve esistere

Validazioni in ItinerarioService

Operazione	Controllo
aggiungItinerario()	Non null, destinazione presente, minPartecipanti ≤ maxPartecipanti
modificaDescrizione()	Itinerario esistente e non prenotato
eliminaItinerario()	Itinerario esistente e non prenotato

10. Messaggi di errore più comuni

Messaggio	Causa	Soluzione
Compilare tutti i campi	Uno o più campi obbligatori sono vuoti	Verificare Nome, Cognome, Classe, Destinazione, Tipo
ID già esistente	L'ID inserito è già presente nel servizio	Usare un ID diverso
Inserire valori numerici validi	Un campo numerico (ID, Età, Giorni, Costo) è difforme rispetto al tipo	Verificare che Età/Giorni, decimale con punto
Itinerario con ID X non trovato	L'ID itinerario nella prenotazione non esiste	Verificare l'ID nel pannello Gestione Itinerari
Alunno con ID X non trovato	L'ID alunno nella lista partecipanti non esiste	Verificare l'ID nel pannello Gestione Alunni
Numero partecipanti troppo basso/alto	Numero di alunni non rispetta i vincoli min/max	Aggiungere o togliere alunni o scegliere un itinerario diverso
Autorizzazione mancante per: [nome]	Un minorenne non ha l'autorizzazione	Registrare l'autorizzazione via AlunnoService.autorizzaAlunno()
Errore caricamento alunni/itinerari	File dati inesistente o corrotto	Il programma continua con lista vuota; il file verrà ricreato alla prossima esecuzione
Impossibile modificare/eliminare l'itinerario	Itinerario già prenotato	Annullare prima le prenotazioni associate
Motivo obbligatorio	Campo motivo vuoto nell'annullamento	Inserire la descrizione del motivo prima di confermare la prenotazione

11. Note tecniche per sviluppatori

Struttura delle classi principali:

Classe	Responsabilità principale
Alunno.java	Entità alunno: id, nome, cognome, età, classe, annoCorso, minorenne (auto), autorizzazione, rinunciaMedica
Destinazione.java	Luogo di destinazione con nome, regione, nazione; ID autoincrementale con contatore statico
Itinerario.java	Dettaglio viaggio: destinazione, giorni, tipo, descrizione, costo, min/maxPartecipanti, optional, prenota
Prenotazione.java	Associa un itinerario a una lista di alunni partecipanti; calcola costo totale e acconto; gestisce annullamento
Classe.java	Raggruppa alunni e prenotazioni di una classe scolastica (usata come contenitore, non direttamente

Classe	Responsabilità principale
AlunnoService.java	CRUD alunni: aggiungi, cerca per ID, rimuovi, autorizza, rinuncia medica
ItinerarioService.java	CRUD itinerari: aggiungi, cerca per ID/destinazione, modifica descrizione, elimina (con protezione
PrenotazioneService.java	Crea prenotazione con validazioni, cerca, annulla, rimuovi alunno, ricalcolo costo
FileManager.java	Serializzazione/deserializzazione Java per alunni, itinerari e prenotazioni tramite ObjectOutputStream
PanelAlunni.java	Pannello Swing: form inserimento alunno + area output elenco; usa AlunnoService
PanelItinerario.java	Pannello Swing: form inserimento itinerario + area output elenco; usa ItinerarioService
PanelPrenotazione.java	Pannello Swing: form prenotazione + annullamento + area output; usa tutti e tre i service
MainFrame.java	JFrame principale: BorderLayout con menu laterale (GridLayout 4×1) e CardLayout centrale
Main.java	Entry point: inizializza service, carica dati, registra shutdown hook, avvia GUI su EDT (SwingUtilities

Dipendenze tra classi (flusso dati):

1. Main istanzia AlunnoService, ItinerarioService, PrenotazioneService
2. FileManager popola i service con i dati deserializzati dai file .dat
3. MainFrame riceve i tre service e li passa ai rispettivi Panel
4. PanelPrenotazione riceve tutti e tre i service per collegare alunni e itinerari alle prenotazioni
5. Alla chiusura, lo shutdown hook in Main legge le liste aggiornate dai service e le salva tramite FileManager

Note sulla serializzazione:

Alunno, Itinerario, Prenotazione e Destinazione implementano java.io.Serializable. Classe.java non implementa Serializable (non è serializzata direttamente). La classe Destinazione usa un contatore statico per gli ID: alla deserializzazione il contatore riparte da 1, quindi è possibile generare ID destinazione duplicati in sessioni diverse. Per evitare conflitti si raccomanda di non fare affidamento sull'ID di Destinazione come chiave primaria.

12. Informazioni sul prodotto

Campo	Descrizione
Prodotto	Gestione Gite Scolastiche
Versione	1.0
Sviluppato da	MySoft snc
Autori codice	Sbragi Niccolò, Barbagli Marco, Bucsai Alex, Bruschi Lorenzo
Anno	2026
Licenza	Tutti i diritti riservati – MySoft snc

